

В данном уроке мы научимся работать с **таймерами** в JavaScript. Разобравшись с таймерами, вы сможете автоматически выполнять на странице какие-либо операции через заданный промежуток времени. К примеру, можно будет сделать слайдер картинок, в котором картинки будут меняться каждую секунду.

Для работы с таймерами в JavaScript используется метод **setInterval**, который запускает заданный код через определенные промежутки времени.

Метод setInterval

Метод **setInterval** работает следующим образом: первым параметром она принимает **имя функции** (без кавычек и круглых скобок), а вторым параметром - через какой **промежуток** запускать эту функцию. Вторым параметром задается в **миллисекундах** (1000 миллисекунд = 1 секунда).

Пример: `window.setInterval(timer, 1000)` - этот код будет запускать функцию **timer** раз в секунду. И каждую секунду эта функция **timer** будет выполнять какие-то полезные операции, например, увеличивать счетчик на странице.

Давайте рассмотрим нужный для этого код. Пусть у нас дан инпут. Будем каждую секунду увеличивать значение его атрибута **value** на единицу:

```
<input type="text" value="1" id="test">
<input type="submit" onclick="start()">

function start() {
    //По нажатию на кнопку наш таймер начнет работать:
    window.setInterval(timer, 1000);
}

//Эта функция будет запускаться каждую секунду
function timer() {
    var elem = document.getElementById('test');
    elem.value = parseInt(elem.value)+1; //parseInt преобразует строку в число
}
```

Обратите внимание на функцию [parseInt](#) - она преобразует строку из атрибута **value** в число.

Это нужно, так как атрибут всегда отдает строку, даже если там хранится число, как у нас, то есть `elem.value` вернет **'1'**, а не **1** (в самом начале таймера, когда в атрибуте еще **1**). И получится, что `elem.value + 1` это **'1'+1**, что дает **'11'**, а не **2**.

Поэтому, если не применять **parseInt** - скрипт начнет складывать наши числа **как строки** и мы увидим, что в инпуте сначала будет **1**, потом **11**, потом **111** и так далее.

Если же написать **parseInt**, то в инпуте будет сначала **1**, потом **2**, потом **3** и так далее.

Остановка таймера

Вы уже знаете, как запустить таймер, давайте теперь научимся его останавливать. Для этого используется метод **clearInterval**, который принимает **уникальный номер** того таймера (созданного через **setInterval**), который нужно остановить.

Давайте посмотрим откуда берется этот номер:

```
<input type="submit" onclick="test()">

function test() {
    timerId = window.setInterval(timer, 1000);
    alert(timerId); //выведет номер таймера
}

//Ничего не делает, просто заглушка
function timer() {

}
```

То есть при создании таймера через **setInterval** мы можем узнать его номер, и потом передать его методу **clearInterval**, чтобы таймер **остановился**.

В следующем примере демонстрируется работа с **clearInterval**. Обратите внимание на то, что запуск и остановка таймера - это разные функции. Чтобы передать номер таймера из одной функции в другую я использую **глобальные переменные** (свойства объекта window), которые мы с вами уже разбирали раньше:

```
<input type="text" value="1" id="test">
<input type="submit" value="start" onclick="start()">
<input type="submit" value="stop" onclick="stop()">

//Эта функция запускает таймер
function start() {
    window.timerId = window.setInterval(timer, 1000);
}

//Эта функция останавливает таймер
function stop() {
    window.clearInterval(window.timerId);
}

//Эта функция меняет value для инпута
function timer() {
    var elem = document.getElementById('test');
    elem.value = parseInt(elem.value)+1;
}
```

Запустите этот код и понажимайте на кнопки. Кнопка *start* будет **запускать** таймер, а кнопка *stop* - **останавливать**. Причем таймер можно запускать и останавливать **много раз** - счетчик в инпуте продолжит с того места, где он остановился. Только в **window.timerId** при каждом новом запуске будет лежать **новое** значение.

Если еще один нюанс - если много раз понажимать на кнопку *start*, не нажимая *stop*, то вы запустите **много таймеров**, каждый из которых будет менять содержимое инпута.

И получится так, что значения атрибута **value** будут меняться не раз в секунду, а гораздо **чаще** (и неравномерно). При этом кнопка *stop* сможет остановить **только последний** таймер (так как каждый новый таймер будет затирать глобальную переменную **window.timerId** и там всегда будет лежать номер последнего запущенного таймера).

Метод setTimeout

Следующий метод, который нам нужен, называется **setTimeout**. Он позволяет сделать **задержку** перед запуском кода (эта задержка случится только **один раз** и код выполнится только один раз, в отличие от **setInterval**).

Вторым параметром **setTimeout** принимает **задержку** (в миллисекундах) перед запуском функции, которая задана первым параметром, например так: `window.setTimeout(func, 3000)` - по сути эта строчка вызывает функции **func**, однако эта функция сработает не сразу, а через **3** секунды.

Давайте сделаем так, чтобы **alert** сработал через **3** секунды после нажатия на кнопку:

```
<input type="submit" onclick="start()">

function start() {
    window.setTimeout(func, 3000);
}

function func() {
    alert('!');
}
```

Что происходит в этом примере - по нажатию на кнопку сработает функция **start**, которая вызывает функцию **func** с задержкой с **3** секунды (**3000** миллисекунд = **3** секунды). Ну, а функция **func** выводит **alert** в тот момент, когда вызвана.

Метод setTimeout и рекурсия

Хотя метод **setTimeout** и не предназначен для создания таймеров, однако их все равно можно делать с ее помощью, если воспользоваться **рекурсией** (это когда функция вызывает сама себя).

В данном примере значение атрибута **value** каждую секунду увеличивается на единицу. Сделано это хитрым образом: нажатие на кнопку *start* запускает функцию **timer**, которая увеличивает значение инпута на **1** и после этого вызывает саму себя с задержкой в **1** секунду. Получится, что через **1** секунду функция **timer** опять начнет работать, опять увеличит значение инпута и опять вызовет **саму себя** с задержкой. И так до бесконечности:

```
<input type="text" value="1" id="test">
<input type="submit" value="start" onclick="timer()">

function timer() {
    var elem = document.getElementById('test');
    elem.value = parseInt(elem.value)+1;

    window.setTimeout(timer, 1000); //рекурсия
```

```
}
```

Остановить такой таймер можно просто не дав запуститься методу **setTimeout**, например, через **if**.

В следующем примере таймер **остановится**, когда значение инпута станет равно **10-ти**:

```
<input type="text" value="1" id="test">
<input type="submit" value="start" onclick="timer()">
function timer() {
    var elem = document.getElementById('test'); //получаем наш инпут
    elem.value = parseInt(elem.value)+1; //увеличиваем значение атрибута на единицу

    if(elem.value < 10) { //если в инпуте меньше 10-ти - то работаем дальше
        //Функция timer запускает саму себя с задержкой в 1 секунду:
        window.setTimeout(timer, 1000);
    }
}
```

Примеры работы с таймерами джаваскрипт, вставьте эти примеры и посмотрите что будет

1.

```
<html
>
    <head>
    <meta charset="utf-8">
    <title> </title>
    <style>
    input {
    width: 200px;
    }
    #wrapper, #test {
    text-align: center;
    }
    #test {
    font-size: 24px;
    }
    </style>
    <script>
    //Запускает таймер
    function go(){
```

```

window.setInterval(timer, 1000);
}
function timer(){
var test = document.getElementById('test');
test.innerHTML = parseInt(test.innerHTML)+1;
}
</script>
</head>
<body>
<div id="wrapper">
<p id="test">
0
</p>
<input type="submit" value="Нажмите на меня!" onclick="go()">
</div>
</body>
</html>

```

2. давайте улучшим таймер и добавим кнопку остановки

```

<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title> </title>
<style>
input {
width: 200px;
}
#wrapper, #test {
text-align: center;
}
#test {
font-size: 24px;
}
</style>
<script>
//Запускает таймер
function go(){
window.timerId = window.setInterval(timer, 500);
document.getElementById('go').disabled = true;
document.getElementById('stop').disabled = false;
}
//Останавливает таймер

```

```

function stop(){
window.clearInterval(window.timerId);
document.getElementById('go').disabled = false;
document.getElementById('stop').disabled = true;
}
function timer(){
var test = document.getElementById('test');
test.innerHTML = parseInt(test.innerHTML)+1;
}
</script>
</head>
<body>
<div id="wrapper">
<p id="test">
0
</p>
<input type="submit" value="Запустить таймер!" onclick="go()" id="go">
<input type="submit" value="Остановить таймер!" onclick="stop()" id="stop" disabled>
</div>
</body>
</html>

```

3. Используя даты мы можем вывести текущее время

```

<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title> </title>
<style>
#wrapper, #clock {
text-align: center;
}
#clock {
font-size: 40px;
margin: 0;
}
</style>
<script>
//Запускает таймер
function go(){
window.timerId = window.setInterval(timer, 500);
}
function timer(){

```

```

var clock = document.getElementById('clock');
var date = new Date();
clock.innerHTML =
addZero(date.getHours())+':'+addZero(date.getMinutes())+':'+addZero(date.getSeconds());
}
function addZero(num){
if(num <=9) return '0'+num;
else return num;
}
</script>
</head>
<body onload="go()">
<div id="wrapper">
<p id="clock">
00:00:00
</p>
</div>
</body>
</html>

```

4. Создадим таймер который будет считать в обратную сторону

```

<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8">
<title> </title>
<style>
input {
width: 200px;
}
#wrapper, #test {
text-align: center;
}
#test {
font-size: 24px;
}
</style>
<script>
//Запускает таймер
function go(){
window.timerId = window.setInterval(timer, 300);
document.getElementById('go').disabled = true;

```

```

}
//Останавливает таймер
function stop(){
window.clearInterval(window.timerId);
}
function timer(){
var test = document.getElementById('test');
var number = parseInt(test.innerHTML)-1;
test.innerHTML = number;
if(number == 0) {
stop();
var timetStop = document.getElementById('stop');
timetStop.innerHTML = 'Обратный отсчет завершен.';
}
}
</script>
</head>
<body>
<div id="wrapper">
<p id="test">
10
</p>
<p id="stop"></p>
<input type="submit" value="Запустить таймер!" onclick="go()" id="go">
</div>
</body>
</html>

```

5. Создадим простой слайдер, для начала сделаем так чтобы у нас изменялись картинки. Скопируйте код ниже и создайте отдельную папку с названием для картинок. Ниже я пометил красным где вам нужно изменить путь к картинкам. Картинки просто назовите от 1 до 3 или можете больше. Если все правильно сделаете, то картинки будут сменяться на другие. Внимательно изучите пример, позже будете улучшать этот пример

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>

  <meta charset="utf-8">

  <title> </title>

  <style>

    #wrapper, #test {

      text-align: center;

    }

  </style>

  <script>

    //Запускает таймер

    function go(){

      window.timerId = window.setInterval(timer, 1000);

    }

    function timer(){

      if(window.number == undefined || window.number == 3)
{//картинки от 1 до 10

        window.number = 1;

      } else window.number = window.number + 1;

      var img = document.getElementById('img');

      img.src = 'pictures/'+window.number+'.png'

    }

  </script>

</head>

<body onload="go()">

  <div id="wrapper">

  </div>

</body>
```

</html>

6. Улучшим наш пример и сделаем автоматический слайдер(карусель) .вставьте код ниже и попробуйте разобраться.

<!DOCTYPE html>

<html>

<head>

 <meta charset="utf-8">

 <title> </title>

 <style>

 input {

 width: 200px;

 }

 #wrapper, #test {

 text-align: center;

 }

 #test {

 font-size: 24px;

 }

 </style>

 <script>

 //Запускает таймер

 function go(){

 window.timerId = window.setInterval(timer, 1000);

 document.getElementById('go').disabled = true;

 }

 function timer(){

```

        var img1 = document.getElementById('img1');
        var img2 = document.getElementById('img2');
        var img3 = document.getElementById('img3');

        var tmp = img1.src;

        img1.src = img2.src;

        img2.src = img3.src;

        img3.src = tmp;
    }
</script>
</head>
<body>
    <div id="wrapper">
        
        
        
        <p>
            <input type="submit" value="Запустить слайдер!"
onclick="go()" id="go">
        </p>
    </div>
</body>
</html>

```

Задания

**Улучшите карусель выше, добавьте
вверх пару изображений и добавьте
вниз два. И кнопку для остановки**

**карусели.То есть будет у вас 2 кнопки
запуск и остановка. Если опять нажмете
на запуск , то слайдер возобновится и
будет работать**

**2. Попробуйте сделать слайдер, но уже
на 90% ширины экрана , пусть он будет
так же автоматический и адаптивный**

**3. сделайте слайдер , чтобы можно было
переключаться между элементами(
картинками) самостоятельно. Например
по кнопке или по нажатию на слайд.**

**P.S мы рассмотрели самые основные
темы по языку джаваскрипт. Мы
рассмотрели то, что поможет вам
оживить страничку. К сожалению часов
не так много чтобы показать все...но я
оставлю полезные материалы для
дальнейшего изучения. База у вас есть,**

будет легче изучать. На этом ресурсе пройдите джаваскрипт все уроки, может станет понятнее :

<https://www.freecodecamp.org/learn>

Здесь вы можете изучить js, здесь вы найдете все о js <https://learn.javascript.ru/>

Здесь можете почитать про события если не очень понятно

<https://itchief.ru/javascript/browser-events#animation-events>

А здесь потренироваться и проверить свои силы : <https://daruse.ru/zadaniya-po-js-dlya-prokachki>

Задачи по всему , что мы проходили

- 1 Сделайте калькулятор, который будет находить площадь и периметр квадрата.
- 2 Напишите скрипт, который будет находить корни квадратного уравнения. Для этого сделайте 3 инпута, в которые будут вводиться коэффициенты уравнения.
- 3 Дан textarea. Пусть в него вводится текст. Сделайте так, чтобы по потере фокуса под текстарееа вывелось сообщение о том, сколько в этом тексте слов.
- 4 Сейчас мы с вами реализуем игру *угадай число*. В этой игре компьютер загадывает число от 1 до 100. В инпут на экране игрок вводит число от 1 до 100, пытаясь угадать, что же загадал компьютер.
Если игрок ввел число, меньше загаданного, компьютер должен написать 'введите число побольше', а если введено больше загаданного, то, соответственно, компьютер должен написать 'введите число поменьше'.
- 5 Давайте реализуем калькулятор. Он будет работать как обычный классический калькулятор: у вас будет инпут, а под ним кнопки с цифрами и знаками операций. По нажатию на цифры они должны появляться в инпуте. По нажатию на знак операции число из инпута должно исчезать. После этого при вводе следующего числа в инпут и нажатия на знак равно в инпуте должен появиться результат операции.
- 6 Выведите в виде списка ul все числа текущего месяца. Средствами CSS поставьте пункты списка в ряд.
- 7 Сейчас мы с вами сделаем сайт, который будет выдавать предсказания. Пусть на этом сайте будет кнопка, по нажатию на которую будет запускаться

таймер, который будет каждые 0.1 секунд выводить в в какой-нибудь див случайное число от 1 до некоторого максимального.

Под дивом пусть будет другая кнопка, по нажатию на которую пользователь нашего сайта может остановить таймер и зафиксировать некоторое число в диве. Это число будет номером предсказания. После этого покажите пользователю предсказание с этим номером, а все лишние кнопки уберите с экрана, чтобы пользователь не мог получить еще одно предсказание. То есть на один заход на сайт - одно предсказание.

Вот образец того, что должно получиться:

?

узнать свою судьбу

При нажатии на кнопку у вас начинается
рандомно меняться число то.есть в
данном случае это предсказание

3

получить предсказание

и при клике на кнопку выводится
рандомное предсказание

Верстку можете использовать эту

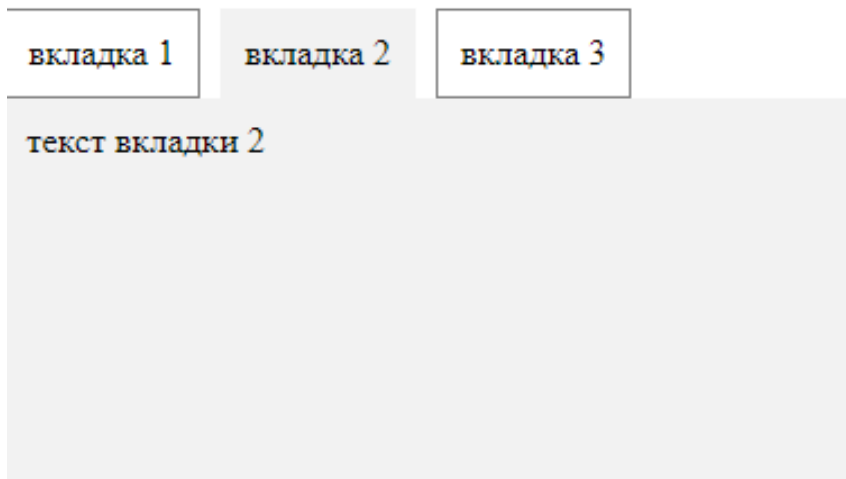
```
<div id="parent">
  <div id="timer">?</div>
  <div id="text"></div>
  <button id="start" class="active">узнать свою судьбу</button>
  <button id="stop">получить предсказание</button>
</div>#parent {
  text-align: center;
}
#parent > * {
  margin-bottom: 10px;
}

#timer {
  font-size: 18px;
}
#text {
  font-size: 17px;
  font-style: italic;
}

button {
  padding: 5px 10px;
  font-size: 15px;
  cursor: pointer;
}
button:not(.active) {
  display: none;
}
```

8 Задача на вкладки

Сейчас мы с вами реализуем вкладки. Сразу посмотрите на образец того, что от вас требуется:



зучите также верстку, на которой мы будем решать задачу:

```
<div id="parent">  
  <div class="menu">  
    <a href="" class="active">вкладка 1</a>  
    <a href="">вкладка 2</a>  
    <a href="">вкладка 3</a>  
  </div>
```

```

<div class="tabs">
  <div class="tab active">
    текст вкладки 1
  </div>
  <div class="tab">
    текст вкладки 2
  </div>
  <div class="tab">
    текст вкладки 3
  </div>
</div>
#parent {
  margin: 0 auto;
  width: 400px;
}

.menu a {
  margin-right: 5px;
  display: inline-block;
  padding: 10px;
  color: black;
  text-decoration: none;
  border: 1px solid gray;
}

.menu a:hover {
  background: #f8f8f8;
}

.menu a.active {
  border-color: transparent;
  background: #f2f2f2;
}

.tabs .tab:not(.active) {
  display: none;
}

.tabs .tab {
  padding: 10px;
}

```

```
background: #2f2f2f;  
height: 300px;  
}
```

Обратите внимание на то, что активная ссылка и активная вкладка имеют класс `active`.

Обсуждение

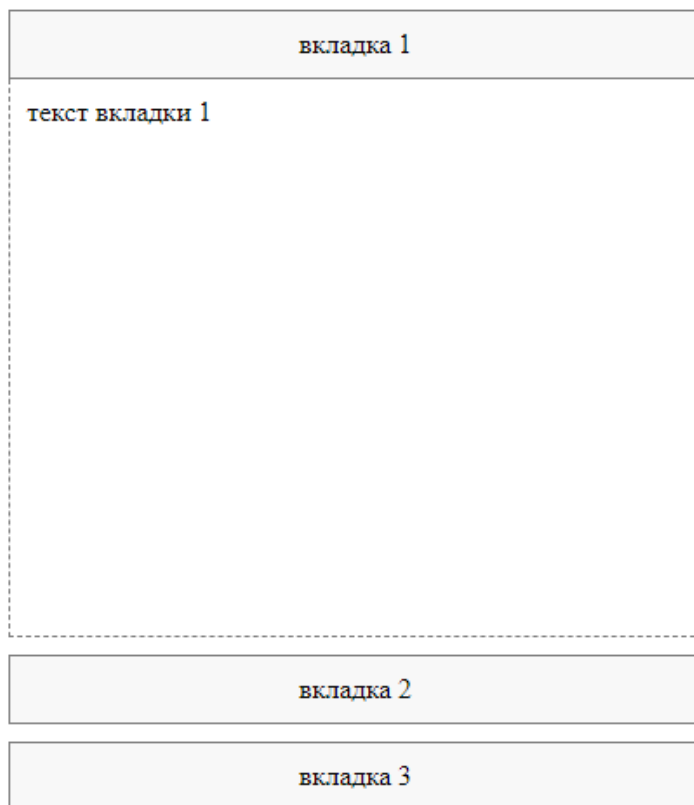
Давайте теперь обсудим то, как решать эту задачу. Очевидно, что нужно как-то связать ссылки с соответствующими им вкладками. В данном [уроке](#) из теоретического учебника мы уже разбирали возможные варианты.

В данном случае я предлагаю связать ссылки и вкладки просто по позиции: первая ссылка будет открывать первую вкладку, вторая ссылка - вторую вкладку и так далее.

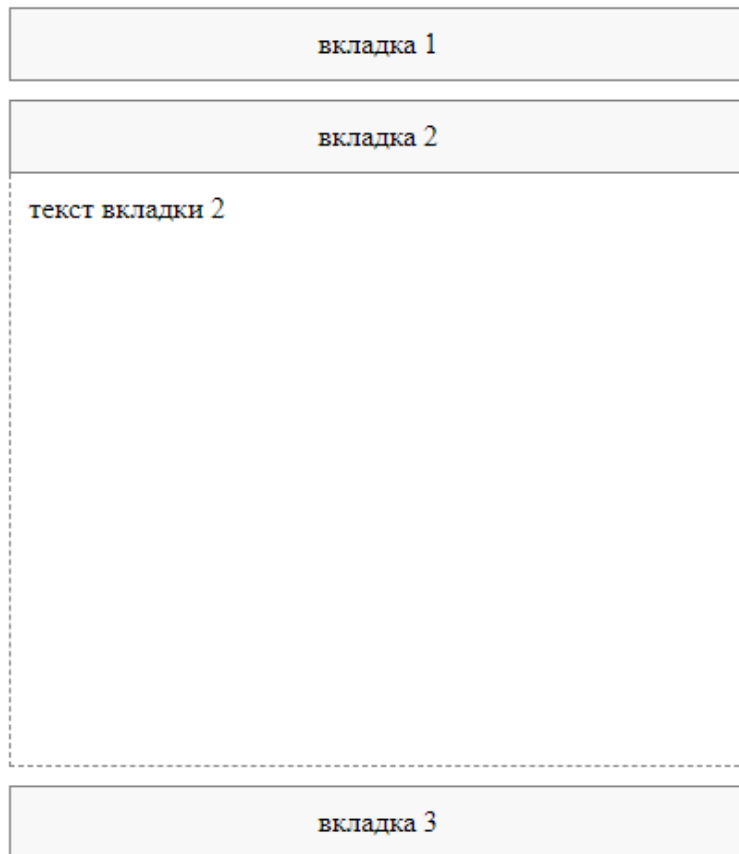
Общий алгоритм будет выглядеть так: по нажатию на кнопку мы должны найти ссылку с классом и убрать ей этот класс. Аналогичным образом мы должны найти вкладку с этим классом и тоже убрать ей этот класс.

Затем мы должны взять ссылку, на которую был клик и активировать ее. Затем мы должны взять вкладку, которая имеет такой же номер, как и наша ссылка - и тоже активировать ее.

9. Давайте теперь реализуем аккордион. Он похож на вкладки, перевернутые на бок. При этом в аккордеоне можно закрыть текущую вкладку, если нажать на активную ссылку.



То есть при клике на другую вкладку ,
предыдущая закрывается и открывается
другой блок, где может быть ваш
контент, например текст или фото



Изучите также верстку, на которой мы будем решать задачу:

```
<div id="parent">
  <div class="tab active">
    <div class="link">
      <a href="#" >вкладка 1</a>
    </div>
    <div class="text">
      текст вкладки 1
    </div>
  </div>
  <div class="tab">
    <div class="link">
      <a href="#" >вкладка 2</a>
    </div>
    <div class="text">
      текст вкладки 2
    </div>
  </div>
</div>
```

```

    <div class="tab">
        <div class="link">
            <a href="#" >вкладка 3</a>
        </div>
        <div class="text">
            текст вкладки 3
        </div>
    </div>
</div>#parent {
    margin: 0 auto;
    width: 400px;
}
.tab {
    margin-bottom: 10px;
}

.tab .link a {
    display: block;
    padding: 10px;
    color: black;
    text-decoration: none;
    text-align: center;
    background: #f8f8f8;
    border: 1px solid gray;
}

.tab .link:hover a {
    background: #f2f2f2;
}

.tab:not(.active) .text {
    display: none;
}

.tab .text {
    border: 1px dashed gray;
    border-top: none;
}

```

```
padding: 10px;  
height: 300px;  
}
```

Обсуждение

Технически HTML код аккордеона удобнее, чем код вкладок: в нем ссылки расположены непосредственно над спойлером, который они открывают и закрывают. Кроме того, класс `active` в нашей верстке дается общему родителю ссылки и спойлера.

Это значит, что при нажатии на ссылку нужно просто деактивировать старую вкладку и активировать новую. Чтобы деактивировать старую вкладку нужно найти вкладку с классом `active` и удалить из нее этот класс.

Чтобы активировать новую вкладку нужно по клику на ссылку найти вкладку-родителя этой ссылки и дать ей класс `active`. Технически такой поиск можно сделать так: просто найти ближайшего родителя ссылки, имеющего класс `tab`.

10

В данном разделе мы с вами реализуем калькулятор продуктов. Он будет представлять собой таблицу, в которую пользователь нашего сайта будет вносить свои покупки.

Пусть покупки вносятся с помощью формы. Кроме того, для каждого продукта предусмотрим ссылку на удаление. Сделаем также возможность редактирования названия, цены и количества продукта. Пусть такое редактирование будет происходить по двойному клику по ячейке таблицы.

Пусть под таблицей выводится суммарная стоимость продуктов. Сделаем так, чтобы эта сумма пересчитывалась при удалении и при редактировании продуктов.

Вот образец того, что должно получиться:

название	цена	количество	добавить
----------	------	------------	----------

Таблица продуктов:

название	цена	кол-во	сумма	удалить
----------	------	--------	-------	---------

общий итог: 0

Хлеб	35	13	добавить
------	----	----	----------

Таблица продуктов:

название	цена	кол-во	сумма	удалить
Молоко	100	3	300	удалить
Хлеб	35	13	455	удалить

общий итог: 755

После добавления товара он добавляется в таблицу , где есть возможность удалить товар и так же считается общая стоимость покупки

Вот верстка, которую вы можете использовать при решении задачи:

```
<div id="parent">
  <div id="form">
    <input id="name" placeholder="название">
    <input id="price" placeholder="цена">
    <input id="amount" placeholder="количество">
    <input id="add" type="button" value="добавить">
  </div>

  <h2>Таблица продуктов:</h2>
  <table id="table">
    <tr>
      <th>название</th>
      <th>цена</th>
      <th>кол-во</th>
      <th>сумма</th>
    </tr>
  </table>
</div>
```



```
border: 1px solid black;
}
#table td.remove {
  color: blue;
  cursor: pointer;
  text-decoration: underline;
}
#table td.remove:hover {
  text-decoration: none;
}

#result {
  text-align: right;
  margin-right: 10px;
}
```

Сразу давайте получим ссылки на все нужные теги в переменные:

```
let name = document.querySelector('#name');
let price = document.querySelector('#price');
let amount = document.querySelector('#amount');
let add = document.querySelector('#add');
let table = document.querySelector('#table');
let total = document.querySelector('#total');
```

Заключение

Это далеко не все

ВОЗМОЖНОСТИ ЯЗЫКА, ЭТО

ОСНОВЫ ЧТО РЕАЛЬНО МОЖНО

сделать используя данный язык. Вы можете влиять на любые теги и стили, воздействовать на кнопки и писать свое поведение элементов на странице.

Вы можете придумать любую идею для реализации на странице и сможете ее воплотить в жизнь, конечно изучение языка на этом не заканчивается. Там очень много всего, но я показал самое основное и простое. Удачи вам с

изучением, ресурсов куча для этого. К сожалению это последняя практическая и больше js не будет, но не стоит расстраиваться, ресурсы для тренировки и изучения я дал.